BLOCK 21

Repetition with for Loops

Process many values using one block of code.

“Do this for every item.”

→

```
for record in records:
    print(record)
```

→

The same logic works for 4 records, 400 records, or 4 million.

Today’s mission

Use loops with counters, accumulators, and conditions to summarize data.

BLOCK 21

Why loops matter

Manual repetition does not scale.

```
print(records[0])
print(records[1])
print(records[2])
print(records[3])
```

```
for record in records:
    print(record)
```

Works for 4 items. Painful for 400.

One instruction pattern for any collection size.

BLOCK 21

Basic for loop

A loop processes one item at a time.

```
temperatures = [72, 75, 81, 69]

for temperature in temperatures:
    print(temperature)
```

Output

```
72
75
81
69
```

Loop variable

temperature temporarily refers to the current item.

BLOCK 21

Loop mental model

Fetch one value, run the body, repeat.

Continue until there are no values left.

```
for item in collection:
    process(item)

print("Loop complete")
```

BLOCK 21

Anatomy of a for loop

Keyword, loop variable, iterable, colon, indented body.

```
for temperature in temperatures:
    print(temperature)
```

keyword

loop variable

iterable

colon

indented body

Iterable = something Python can step through.

A list is the most common beginner example.

BLOCK 21

Loop variables change each iteration

The name stays the same; the referenced item changes.

```
scores = [82, 91, 77]

for score in scores:
    print(score)
```

Iteration 1
score = 82

Iteration 2
score = 91

Iteration 3
score = 77

Think: “current score”

BLOCK 21

Indentation and execution order

Indented lines are inside the loop.

```
for temperature in temperatures:
    print(temperature)
    print("Processed")

print("All temperatures processed")
```

Inside loop

- print current value
- print “Processed”
- repeat for every item

After loop

The final print runs once, after all items are processed.

BLOCK 21

Using range()

range() generates a sequence of numbers.

```
for number in range(5):
    print(number)
```

```
0
1
2
3
4
```

The ending value is excluded.

range(5) means
0 through 4

Useful for fixed counts, numbered steps, and test loops.

BLOCK 21

Different ranges

Start, stop, and step.

```
for number in range(1, 5):
    print(number)
```

1, 2, 3, 4

```
for number in range(0, 10, 2):
    print(number)
```

0, 2, 4, 6, 8

range(start, stop, step)

Stop is still excluded.

BLOCK 21

Counters

A counter tracks how many times something happens.

```
count = 0

for record in records:
    count = count + 1

print(count)
```

Start at zero

Add one per item

Print final count

```
count += 1
```

Shortcut for
count = count + 1

BLOCK 21

Accumulators

An accumulator builds a running total.

```
values = [10, 20, 30]

total = 0

for value in values:
    total += value

print(total)
```

After 10
total = 10

After 20
total = 30

After 30
total = 60

Final answer: 60

BLOCK 21

Calculating an average

Total first, divide after the loop.

```
values = [10, 20, 30]

total = 0

for value in values:
    total += value

average = total / len(values)
print(average)
```

Pattern

1. Initialize total

2. Accumulate inside loop

3. Divide after loop

Built-ins like sum() exist,
but this teaches the pattern.

BLOCK 21

Condition inside a loop

Loops and if statements work together.

```
scores = [82, 61, 74, 95, 55, 88]

passing_count = 0

for score in scores:
    if score >= 70:
        passing_count += 1

print(passing_count)
```

For each score...

Check the condition

Only increment when True

Nested indentation matters.

BLOCK 21

Finding minimum and maximum conceptually

Compare each item against the best value seen so far.

```
values = [12, 8, 31, 4]

smallest = values[0]

for value in values:
    if value < smallest:
        smallest = value
```

Mental model

- Start with a reasonable first value.
- Check one value at a time.
- Replace the stored answer when a better one appears.

Python has min() and max(),
but this explains how summary logic works.

BLOCK 21

Guided exercise 1: print every value

Start with the simplest loop.

```
scores = [82, 91, 77, 88, 95]

for score in scores:
    print(score)
```

Tasks

- Type the code.
- Predict the output.
- Run it.
- Change one score and run again.

Exercise time: 4 minutes

BLOCK 21

Guided exercise 2: count values

Build the counter pattern manually.

```
records = ["EQ-101", "EQ-102", "EQ-103"]

count = 0

for record in records:
    count += 1

print(f"Records: {count}")
```

Why not just len(records)?

Because the counter pattern is used later when only some items should count.

Exercise time: 4 minutes

BLOCK 21

Guided exercise 3: total storage

Use an accumulator for file sizes.

```
file_sizes = [12.5, 8.2, 4.7, 31.0]

total_storage = 0

for file_size in file_sizes:
    total_storage += file_size

print(f"Total MB: {total_storage:.1f}")
```

Accumulator

total_storage remembers the running total.

Exercise time: 5 minutes

BLOCK 21

Guided exercise 4: average file size

Combine total, length, and formatted output.

```
file_sizes = [12.5, 8.2, 4.7, 31.0]

total_storage = 0

for file_size in file_sizes:
    total_storage += file_size

average_size = total_storage / len(file_sizes)

print(f"Average MB: {average_size:.1f}")
```

Ask before running

- What is the total?
- How many values are there?
- When should we divide?
- Why after the loop?

Exercise time: 5 minutes

Guided exercise 5: count passing scores

Only count values that meet a condition.

```
scores = [82, 61, 74, 95, 55, 88]

passing_count = 0

for score in scores:
    if score >= 70:
        passing_count += 1

print(f"Passing: {passing_count}")
```

Nested condition

Loop: every score
If: only passing scores

Exercise time: 5 minutes

BLOCK 21

Guided exercise 6: count errors

Multiple summaries from one loop.

```
error_counts = [0, 2, 0, 5, 1, 0]

record_count = 0
records_with_errors = 0
total_errors = 0

for error_count in error_counts:
    record_count += 1
    total_errors += error_count
    if error_count > 0:
        records_with_errors += 1
```

Calculate

- Number of records
- Number with errors
- Total number of errors

One pass through the data can produce several summaries.

BLOCK 21

Applied challenge: completion summary

Process a list of completion percentages.

```
completion_values = [100, 82, 97, 45, 100, 68, 91]
```

Calculate

- Total number of records
- Number complete
- Number incomplete
- Number at least 90% complete
- Sum of all percentages
- Average completion percentage

Required patterns

At least one counter

At least one accumulator

At least one if inside the loop

Challenge time: 12 minutes

BLOCK 21

Applied challenge scaffold

Initialize first, loop second, calculate final values last.

```
total_records = 0
complete_count = 0
incomplete_count = 0
at_least_90_count = 0
percentage_sum = 0

for completion in completion_values:
    total_records += 1
    percentage_sum += completion

    # add your conditions here

average_completion = percentage_sum / total_records
```

Build in layers

1. Count every record
2. Add to the total
3. Add conditions
4. Calculate average after loop

BLOCK 21

Common loop mistakes

Most loop bugs are small structure mistakes.

Forgetting indentation

```
for score in scores:
print(score)
```

Calculating too early

```
for value in values:
    total += value
    average = total / len(values)
```

Not initializing

```
for value in values:
    total += value
```

Read loops as a story:

Initialize → repeat → update → finish

Next: loops over structured records.